

SOURCE-BACKED COMPANION

AIRLOCK PROGRAMS

Structured Unix work between
agents and the world.

THE AUTHORIZING RAIL

CONTRACT MODEL

```
1 ActionCall
2   -> PlanDraft
3   -> Admission
4   -> ExecutionAuthority
5   -> Capture | Invoke | Apply | RequestExternal
```

MACOS FIRST / AGENT ONLY

USABLE DEVELOPER PREVIEW / 2026-07-30



HOW TO READ THIS BOOK

These pages show current program syntax without upgrading a developer preview into a shell-replacement claim.

SOURCE / DESIGN.md / README.md / docs/macOS-v1.md

AIRLOCK PROGRAMS EXPRESS STRUCTURED UNIX WORK. EXISTING UNIX PROGRAMS STILL DO THE WORK.

RUNNABLE

Checked-in current syntax or CLI surface. It may still require bindings, policy, fixtures, or a supported macOS profile.

CONTRACT MODEL

Implemented typed architecture, shown as data or a rail. It is not Airlock source syntax.

INTEGRATION SKETCH

Current language forms aimed at real OpenShell or Vouch work. The remote integration has not run end to end.

CAVEAT

A visible boundary on what the example proves. Compatibility is broad, native containment is narrow, and receipts are not omniscience.

01 CAPTURE

OBSERVATION

Information enters.

02 INVOKE

COMPUTATION

Existing code runs.

03 APPLY

MUTATION

Managed state changes.

04 REQUEST EXTERNAL

EMISSION

Intent stages.

THE TWO LAWS

Only the Reaper unlinks. Explicit profiles may narrow authority; agent code cannot widen it.

WHAT IS NOT CLAIMED

No confidentiality claim for native-contained. No VM equivalence. No broad shell replacement claim yet.

INSTALL. PROBE. INSPECT.

Start from a checkout, build the paired command surfaces, then ask Airlock to describe itself.

SOURCE / README.md: Install from a checkout

docs/programs/snippets/14-install.sh

RUNNABLE

```
1 bun install
2 bun run verify
3 bun run build:macos
4 sh scripts/install-macos.sh
5 export PATH="$HOME/.local/bin:$PATH"
6
7 airlock doctor
8 airlock actions
9 airlock schema plan
10 airlock-agent actions
```

SUPERVISOR SURFACE

`airlock` retains policy, raw maintenance, Hold, Outbox commit/cancel, undo, and reap authority.

AGENT SURFACE

`airlock-agent` exposes programs, schemas, capabilities, and read-only state inspection – not terminal authority.

THE HARNESS GIVES THE AGENT AIRLOCK-AGENT, NOT A PEER SHELL TOOL.

EVIDENCE / Current release requirement: macOS and Bun 1.3.11 or newer. The install prefix must be on PATH.

YOUR FIRST AIRLOCK PROGRAM

The executable is one field. Every argument is a separate atom. The runtime receives no command string.

SOURCE / src/actions/Catalog.ts / src/process/Process.ts

docs/programs/snippets/01-first.air

RUNNABLE

```
1 let result = process.run({
2   executable: "/usr/bin/printf",
3   args: ["%s\n", "hello from Airlock"],
4   cwd: workspace,
5   stdin: "discard",
6   stdout: "capture",
7   stderr: "capture",
8   timeoutMs: 5000,
9   cellProfile: "compatibility"
10 })
11
12 assert result.state == "succeeded", "printf failed"
13 return result
```

SUPERVISOR INVOCATION

RUNNABLE

```
1 airlock run hello.air \
2   --workspace /tmp \
3   --bindings '{"workspace":"/tmp"}'
```

NO IMPLICIT SHELL

Airlock invokes `[executable, ...args]`. Metacharacters inside an argument remain literal at this seam.

AMBIENT HOST AUTHORITY

This example selects compatibility. Child reads, writes, sends, helpers, config, and descriptors are not contained.

EVIDENCE / Structured invocation removes shell parsing from orchestration; it does not make compatibility a sandbox.

A SMALL LANGUAGE AROUND EFFECTS

Bindings supply values. Pure control chooses when to request actions. Neither can mint authority.

SOURCE / `src/language/ast.ts` / `src/language/evaluator.ts`

docs/programs/snippets/02-control.air

RUNNABLE

```

1  assert enabled, "the supervisor must enable this workload"
2
3  for entry in writes {
4    let written = file.write({
5      path: entry.path,
6      content: entry.content
7    })
8    assert written.state == "applied", "write failed"
9  }
10
11 if expected == 3 {
12   return { state: "complete", iterations: expected }
13 } else {
14   return { state: "unexpected-input", iterations: expected }
15 }
```

FINITE

List iteration snapshots a finite list. Range loops are half-open and bounded.

TYPED FAILURE

`assert` stops later work and preserves completed action evidence.

NO WIDENING

Bindings change data, never the supervisor-selected profile or grants.

AGENT TOOL CALL

RUNNABLE

```

1  airlock-agent run docs/programs/snippets/02-control.air \
2    --workspace /tmp/airlock-demo \
3    --bindings '{
4      "enabled": true,
5      "expected": 3,
6      "writes": [
7        {"path": "a.txt", "content": "a"},
8        {"path": "b.txt", "content": "b"},
9        {"path": "c.txt", "content": "c"}
10     ]
11   }'
```

EVIDENCE / The evaluator has one injected world seam: `ActionResolver.resolve(action, input)`.

FOUR EFFECTS. ONE CLOSED PLAN.

Actions lower into an implemented candidate Plan algebra before Admission binds any authority.

SOURCE / `src/plan/Plan.ts` / `docs/contracts/plan-runtime.md`

THE CLOSED ALGEBRA

CONTRACT MODEL

```
1 PlanNode =  
2   Capture  
3   | Invoke  
4   | Apply  
5   | RequestExternal
```

01 CAPTURE

`file.read / glob / stat`
Provenance attaches at entry.

02 INVOKE

`process.run`
A selected profile runs existing code.

03 APPLY

`file.write / move / remove`
Hold performs the live transition.

04 REQUEST EXTERNAL

`http.stage`
Outbox retains intent before dispatch.



IMPLEMENTED CANDIDATE SEAM

The four nodes and total lowering exist. Representative completeness across Unix work remains a falsifiable hypothesis.

ONLY TWO LAWS

The Plan algebra is not promoted into a law. The Reaper and ratchet remain the only frozen invariants.

EVIDENCE / A PlanDraft has unresolved requirements. ExecutionAuthority is the Runtime input.

CAPTURE THE WORKSPACE

Read, inspect, list, glob, and stat are explicit observations with resource requirements and artifacts.

SOURCE / src/actions/Catalog.ts / examples/parity/filesystem.air

docs/programs/snippets/03-capture.air

RUNNABLE

```
1 let readme = file.read({
2   path: "README.md",
3   format: "text"
4 })
5
6 let sources = file.glob({
7   root: ".",
8   pattern: "**/*.ts"
9 })
10
11 let metadata = file.stat({
12   path: "README.md"
13 })
14
15 return {
16   readme: readme,
17   sources: sources,
18   metadata: metadata
19 }
```

ACTIONS

5

inspect / read / list / glob /
stat

PLAN NODE

Capture

observation enters

DEFAULT REALM

local

still admitted explicitly

WHAT PROVENANCE BUYS

Downstream artifacts can point back to the admitted locator and resource identity used for capture.

WHAT IT DOES NOT BUY

Native-contained currently permits ambient host reads. Observation evidence is not a confidentiality boundary.

EVIDENCE / Current formats: text, bytes, and JSON. Following symlinks in file.stat is rejected.

INVOKE WITHOUT COMMAND CONSTRUCTION

Literal argv replaces shell parsing at the Airlock boundary – including strings that look hostile to a shell.

SOURCE / examples/parity/process-pipeline.air / src/process/Process.ts

docs/programs/snippets/04-invoke-literal.air

RUNNABLE

```
1 let result = process.run({
2   executable: "/usr/bin/printf",
3   args: [
4     "%s",
5     "literal; touch should-not-exist"
6   ],
7   cwd: workspace,
8   stdin: "discard",
9   stdout: "capture",
10  stderr: "capture",
11  outputLimitBytes: 65536,
12  cellProfile: "compatibility"
13 })
14
15 assert result.state == "succeeded", "printf failed"
16 return result
```

/USR/BIN/PRINTF

executable

>

distinct

["%S", "LITERAL; TOUCH..."]

argument atoms

STRUCTURED STREAMS

stdin is discard, inherit, inline text, or an ArtifactId. stdout and stderr are capture, discard, or inherit.

AMBIENT HOST AUTHORITY

The literal string is safe from implicit shell parsing, but the compatibility child remains broadly empowered.

EVIDENCE / An explicitly admitted interpreter may still interpret its own input. Airlock does not infer intent.

A PIPELINE IS AN ARTIFACT GRAPH

Captured stdout becomes named input to the next Invoke. No pipe syntax or intermediary shell process is required.

SOURCE / examples/parity/process-pipeline.air / test/parity-process.test.ts

docs/programs/snippets/05-pipeline.air

RUNNABLE

```

1 let selected = process.run({
2   executable: "/usr/bin/grep",
3   args: ["-n", "needle", "input.txt"],
4   cwd: workspace,
5   stdin: "discard",
6   stdout: "capture",
7   stderr: "capture",
8   cellProfile: "compatibility"
9 })
10 assert selected.state == "succeeded", "grep failed"
11
12 let upper = process.run({
13   executable: "/usr/bin/tr",
14   args: ["a-z", "A-Z"],
15   cwd: workspace,
16   stdin: {
17     kind: "artifact",
18     id: selected.stdout_artifact.id
19   },
20   stdout: "capture",
21   stderr: "capture",
22   cellProfile: "compatibility"
23 })
24 assert upper.state == "succeeded", "tr failed"
25
26 return {
27   selected: selected.stdout,
28   transformed: upper.stdout
29 }
```

GREP

Invoke

>

stdout

ARTIFACT/SELECTED

>

stdin

TR

Invoke

EXPLICIT DATAFLOW

The artifact id carries bytes and provenance. Process order follows declared dependency. This is not an atomic pipeline.

PROOF RECEIPT

Executed output: `2:needle second` became `2:NEEDLE SECOND`; both displayed Invokes succeeded.

EVIDENCE / Output and the final returned value remain bounded by configured limits.

UNIX PROGRAMS STAY UNIX PROGRAMS

Airlock does not implement archive semantics. It gives `/usr/bin/tar` a structured invocation and bounded authority.

SOURCE / `examples/parity/corpus/archive-roundtrip.air`

docs/programs/snippets/06-tar.air

RUNNABLE

```
1 let packed = process.run({
2   executable: "/usr/bin/tar",
3   args: ["-czf", "package.tgz", "package"],
4   cwd: workspace,
5   env: {
6     COPYFILE_DISABLE: "1",
7     LANG: "C"
8   },
9   stdin: "discard",
10  stdout: "capture",
11  stderr: "capture",
12  timeoutMs: 30000,
13  cellProfile: "native-contained"
14 })
15 assert packed.state == "succeeded", "archive creation failed"
16
17 let archive = file.inspect({
18   path: workspace + "/package.tgz"
19 })
20
21 return { packed: packed, archive: archive }
```

TAR

archive

GIT

versioning

SED

editing

MAKE

build

SQLITE3

database

PRISTINE COMPONENT

The generic action is `process.run`. Tool definitions may improve ergonomics but cannot add authority or Plan constructors.

NATIVE BOUNDARY

Native-contained supports a narrower file/directory workload. Descendant executables must be declared.

EVIDENCE / The archive algorithm remains tar's. Airlock owns admission, execution bounds, finality, and evidence.

APPLY THROUGH HOLD

Managed file actions perform admitted local changes. Hold displaces prior bindings before live state changes.

SOURCE / src/Hold.ts / examples/parity/filesystem.air

docs/programs/snippets/07-apply.air

RUNNABLE

```

1 let directory = file.mkdir({
2   path: "generated",
3   parents: true
4 })
5
6 let written = file.write({
7   path: "generated/report.txt",
8   content: "ready\n"
9 })
10
11 let copied = file.copy({
12   source: "generated/report.txt",
13   destination: "generated/report.backup.txt"
14 })
15
16 let moved = file.move({
17   source: "generated/report.txt",
18   destination: "report.txt"
19 })
20
21 let removed = file.remove({
22   path: "generated/report.backup.txt"
23 })
24
25 return {
26   directory: directory,
27   written: written,
28   copied: copied,
29   moved: moved,
30   removed: removed
31 }
```

LIVE BINDING

rename prior

>

HOLD

retained state

>

REPLACEMENT

install live

LAW 01

Only the Reaper unlinks. Managed removal or replacement first retains prior state through Hold.

RECOVERY BOUNDARY

Undo is bounded by retained material, supported filesystem semantics, and conflict checks. Multi-entry ACID is not claimed.

EVIDENCE / Compatibility-child syscalls do not become Apply and receive no Hold guarantee.

LET RM BE RM – REMOVE FINALITY INSTEAD

The destructive executable runs inside the private view. Its deletion becomes a delta; Hold owns the live transition.

SOURCE / examples/parity/native-destructive.air / test/parity-native-contained.test.ts

docs/programs/snippets/08-rm-rf.air

RUNNABLE

```
1 let removed = process.run({
2   executable: "/bin/rm",
3   args: ["-rf", "protected"],
4   cwd: workspace,
5   stdin: "discard",
6   stdout: "capture",
7   stderr: "capture",
8   timeoutMs: 30000,
9   cellProfile: "native-contained"
10 })
11
12 assert removed.state == "succeeded", "contained removal failed"
13 return removed
```

PRIVATE VIEW

rm -rf executes

>

delta

ADMISSION

revalidate

>

Apply

HOLD

recoverable live change

THE USEFUL CAPABILITY REMAINS. IN THIS SUPPORTED NATIVE-CONTAINED WORKLOAD, THE IRREVERSIBLE PATH DOES NOT.

EXECUTED CORPUS

The destructive native workload removes a directory through the private Cell, merges the delta, and restores it with supervisor undo.

NOT SYSCALL INTERCEPTION

Running `/bin/rm -rf`` in compatibility would retain ambient host mutation authority and sit outside this recovery envelope.

EVIDENCE / This is the clearest example of Airlock physics around an existing Unix tool.

TURN THE RATCHET

Zero configuration stays broad. A supervisor-selected profile may narrow authority; agent code cannot select a weaker profile.

SOURCE / DESIGN.md: The ratchet law / docs/macos-v1.md

COMPATIBILITY

IMPLEMENTED

BROAD ZERO-CONFIG CAPABILITY

Existing host authority, configuration, descendants, descriptors, filesystem, and network remain available. No containment claim.

NATIVE-CONTAINED

IMPLEMENTED

EXPLICIT MACOS NARROWING

Private workspace, live-write denial, network denial, delta generation, and Hold-backed Apply for the supported subset.

VM-ENCLOSED

FUTURE

STRONGER FUTURE ENCLOSURE

Unavailable today. The CLI refuses this profile rather than silently falling back to compatibility.

THE RATCHET IS EXPLICIT

RUNNABLE

```
1 airlock exec \  
2   --executable /bin/echo \  
3   --arg "hello world" \  
4   --cwd /tmp  
5  
6 airlock run create.air \  
7   --workspace /absolute/workspace \  
8   --profile native-contained
```

LAW 02

Restrictions are opt-in. Once selected, missing enforcement fails closed; an Airlock program cannot widen the profile.

NATIVE IS NOT CONFIDENTIAL

The current Seatbelt profile permits ambient host reads and does not claim complete execution closure or VM equivalence.

EVIDENCE / Curation improves precision but never gates compatibility capability.

ADMISSION OWNS AUTHORITY

A supervisor-owned policy binds paths, executable identities, endpoint selectors, profile, and principal.

SOURCE / `src/admission/Admission.ts` / `README.md` quickstart

`docs/programs/snippets/18-policy.json`

RUNNABLE

```
1 {
2   "schemaVersion": "airlock/admission-policy/v1",
3   "profile": "native-contained",
4   "principal": "agent:example",
5   "realm": "local",
6   "admittedBy": "operator:example",
7   "pathAllowlist": ["/absolute/workspace/**"],
8   "executableAllowlist": ["/usr/bin/touch"],
9   "executableEdges": [],
10  "endpointAllowlist": []
11 }
```



SCHEMA-FIRST

The policy, requirements, grants, handles, resolutions, and execution authority are decoded contracts – not CLI folklore.

NO AUTHORITY MINTING

Program source and definitions cannot mint grants. This example policy authorizes only `/usr/bin/touch` in its listed workspace.

EVIDENCE / Runtime revalidates closure and grant lifetime at each node boundary.

COMPUTE IN A PRIVATE VIEW

Native-contained runs the executable away from the live workspace, derives a supported delta, then asks Hold to apply it.

SOURCE / src/cell/Cell.ts / docs/macos-v1.md

docs/programs/snippets/09-native-touch.air

RUNNABLE

```
1 let created = process.run({
2   executable: "/usr/bin/touch",
3   args: ["created.txt"],
4   cwd: workspace,
5   stdin: "discard",
6   stdout: "capture",
7   stderr: "capture",
8   timeoutMs: 30000,
9   cellProfile: "native-contained"
10 })
11
12 assert created.state == "succeeded", "contained create failed"
13 return created
```

docs/programs/snippets/15-native-run.sh

RUNNABLE

```
1 AIRLOCK_POLICY_FILE=/absolute/policy.json \
2   airlock run /absolute/create.air \
3   --workspace /absolute/workspace \
4   --profile native-contained \
5   --bindings '{"workspace":"/absolute/workspace"}'
6
7 airlock held
8 airlock undo
```

FINGERPRINT

live workspace

>

PRIVATE CELL

network denied

>

delta

HOLD APPLY

drift checked

NARROW EVIDENCE ENVELOPE

Current native merge covers supported top-level regular-file or directory deltas. Hardlinks, special files, live foreign writers, ACL/xattr fidelity, and multi-entry atomicity remain outside the claim.

EVIDENCE / Unavailable native enforcement never falls back to compatibility.

STAGE BEFORE THE WIRE

`RequestExternal` creates durable local intent. A later supervisor action decides whether to cancel or dispatch.

SOURCE / `src/Outbox.ts` / `src/outbox/Contract.ts`

docs/programs/snippets/10-http-stage.air

RUNNABLE

```
1 let staged = http.stage({
2   endpoint: endpoint,
3   method: "POST",
4   headers: {
5     "content-type": "application/json"
6   },
7   body: "{\\"operation\\":\\"snapshot-ready\\"}",
8   holdMillis: 300000
9 })
10
11 assert staged.state == "staged"
12 return staged
```

docs/programs/snippets/16-outbox.sh

RUNNABLE

```
1 airlock pending
2
3 # Cancel while the intent is still staged:
4 airlock cancel emi...
5
6 # Or let the supervisor cross the wire:
7 airlock commit emi...
```

STAGED

inert

>

choose

CANCEL

before claim

OR

COMMIT

crosses wire

ONLY COMMIT DISPATCHES

Airlock-owned HTTP intent touches the wire only inside `Outbox.commit`. Node success means staged, not sent.

HONEST UNCERTAINTY

After a possible dispatch, ambiguity becomes ``uncertain``; it is never automatic permission to retry.

EVIDENCE / A direct supervisor commit can cross before `holdUntil`; `flush` is the operation that honors the waiting window.

VOUCH PATH I: SNAPSHOT

OpenShell remains OpenShell. Python and SQLite retain their application semantics. Airlock structures the host orchestration.

SOURCE / examples/vouch/snapshot.air / ../vouch/assets/box-runtime-v1.py

docs/programs/snippets/11-vouch-snapshot.air:1-21

INTEGRATION SKETCH

```
1 let snapshot = process.run({
2   executable: openshell,
3   args: [
4     "sandbox", "exec",
5     "--name", sandbox_name,
6     "--no-tty",
7     "--timeout", "120",
8     "--",
9     "/usr/bin/python3", "-I", "-S",
10    "/opt/airlock-programs/sqlite-safe-snapshot.py",
11    "--state-root", hermes_home,
12    "--output", remote_archive
13  ],
14  cwd: workspace,
15  stdin: "discard",
16  stdout: "capture",
17  stderr: "capture",
18  timeoutMs: 150000,
19  cellProfile: "compatibility"
20 })
21 assert snapshot.exit_code == 0, "snapshot failed"
```

STRUCTURED CONTROLLER CALL

OpenShell is one admitted executable. Its subcommand, sandbox name, timeout, remote executable, and helper arguments stay distinct.

COMPATIBILITY - AMBIENT

OpenShell owns the remote effects; Outbox does not mediate them. Local process cancellation is not remote operation cancellation.

EVIDENCE / Snapshot phase. The existing Python helper retains SQLite and archive semantics.

15B

INTEGRATION SKETCH

VOUCH SNAPSHOT: BRING THE ARTIFACT HOME

A second structured invocation downloads the remote artifact. A managed observation records local archive metadata.

SOURCE / examples/vouch/snapshot.air / examples/vouch/OPERATIONS.md

docs/programs/snippets/11-vouch-snapshot.air:23-40

INTEGRATION SKETCH

```
23 let downloaded = process.run({
24   executable: openshell,
25   args: [
26     "sandbox", "download",
27     sandbox_name,
28     remote_archive,
29     local_archive
30   ],
31   cwd: workspace,
32   stdin: "discard",
33   stdout: "capture",
34   stderr: "capture",
35   timeoutMs: 180000,
36   cellProfile: "compatibility"
37 })
38 assert downloaded.exit_code == 0, "download failed"
39
40 return file.inspect({ path: local_archive })
```

REMOTE ARCHIVE

OpenShell

>

LOCAL ARCHIVE

>

CAPTURE RECEIPT

download

inspect

GENERIC PHYSICS

Invoke existing controller. Capture process evidence. Inspect the result. No Vouch-shaped Airlock primitive appears.

REMOTE PROOF MISSING

The source parses and lowers today. A real OpenShell/Vouch snapshot run remains unproved.

EVIDENCE / Airlock does not claim the remote artifact is correct merely because download exited zero.

16A

INTEGRATION SKETCH

VOUCH PATH II: STAGE RESTORE INPUT

Inspect local evidence first, then upload with a structured controller invocation and an explicit deadline.

SOURCE / examples/vouch/restore.air / examples/vouch/OPERATIONS.md

docs/programs/snippets/12-vouch-restore.air:1-20

INTEGRATION SKETCH

```
1 let archive = file.inspect({ path: local_archive })
2 assert archive.bytes > 0, "restore archive is empty"
3
4 let uploaded = process.run({
5   executable: openshell,
6   args: [
7     "sandbox", "upload",
8     sandbox_name,
9     local_archive,
10    remote_archive,
11    "--no-git-ignore"
12  ],
13   cwd: workspace,
14   stdin: "discard",
15   stdout: "capture",
16   stderr: "capture",
17   timeoutMs: 120000,
18   cellProfile: "compatibility"
19 })
20 assert uploaded.exit_code == 0, "upload failed"
```

PRECONDITION IS VISIBLE

The archive must exist and contain bytes before the upload request can begin.

COMPATIBILITY - AMBIENT

OpenShell owns upload behavior and remote effects. Outbox does not mediate this controller traffic.

EVIDENCE / Local timeout bounds the controller process. It does not prove remote cancellation.

VOUCH RESTORE: EXECUTE THE EXISTING HELPER

The embedded Python command becomes literal argv to an existing helper, with captured streams and an explicit postcondition.

SOURCE / examples/vouch/restore.air / ../vouch/assets/box-runtime-v1.py

docs/programs/snippets/12-vouch-restore.air:22-44

INTEGRATION SKETCH

```
22 let restored = process.run({
23   executable: openshell,
24   args: [
25     "sandbox", "exec",
26     "--name", sandbox_name,
27     "--no-tty",
28     "--timeout", "180",
29     "--",
30     "/usr/bin/python3", "-I", "-S",
31     "/opt/airlock-programs/safe-restore.py",
32     "--archive", remote_archive,
33     "--destination", state_home
34   ],
35   cwd: workspace,
36   stdin: "discard",
37   stdout: "capture",
38   stderr: "capture",
39   timeoutMs: 210000,
40   cellProfile: "compatibility"
41 })
42
43 assert restored.exit_code == 0, "restore failed"
44 return restored
```

WHAT GETS SIMPLER

Executable, argv, deadline, stdout, stderr, and the exit postcondition stay visible instead of hiding inside shell text.

WHAT AIRLOCK DOES NOT OWN

Archive collision policy, SQLite correctness, OpenShell transport, and helper correctness remain existing-program semantics.

EVIDENCE / A remote `/usr/bin/python3` argv atom is not a local descendant executable edge.

17A

INTEGRATION SKETCH

VOUCH PATH III: PRESERVE BEFORE REPLACE

The replacement route begins by running the snapshot program and refusing to continue if that phase fails.

SOURCE / examples/vouch/replace.air / docs/vouch-first.md

docs/programs/snippets/13-vouch-replace.air:1-15

INTEGRATION SKETCH

```
1 let snapshot = process.run({
2   executable: airlock_program,
3   args: [
4     "run", snapshot_program,
5     "--sandbox", sandbox_name,
6     "--archive", local_archive
7   ],
8   cwd: workspace,
9   stdin: "discard",
10  stdout: "capture",
11  stderr: "capture",
12  timeoutMs: 300000,
13  cellProfile: "compatibility"
14 })
15 assert snapshot.exit_code == 0, "snapshot failed"
```

SNAPSHOT PROGRAM

Invoke

>

assert

LOCAL ARCHIVE

captured result

FAILURE IS A STOP

The assertion prevents later backup and staging requests after a failed snapshot result.

STILL AMBIENT

The nested Airlock controller invocation uses compatibility in this sketch. Its internal remote effects are not contained here.

EVIDENCE / This page proves program shape only, not a completed Vouch replacement.

17B

INTEGRATION SKETCH

VOUCH REPLACE: RETAIN, THEN STAGE

Local recovery and remote finality stay separate: retain a backup through Apply, then stage replacement intent.

SOURCE / examples/vouch/replace.air / docs/vouch-first.md

docs/programs/snippets/13-vouch-replace.air:17-35

INTEGRATION SKETCH

```
17 let saved = file.copy({
18   source: local_archive,
19   destination: durable_backup
20 })
21 assert saved.state == "applied", "backup was not retained"
22
23 let replacement = http.stage({
24   method: "POST",
25   endpoint: realm_endpoint,
26   holdMillis: 30000,
27   body: "{\"action\":\"replace-machine\"}"
28 })
29 assert replacement.state == "staged"
30
31 return {
32   snapshot: snapshot,
33   saved: saved,
34   replacement: replacement
35 }
```



THIS PROGRAM DOES NOT REPLACE ANYTHING

There is no commit, completion check, health gate, restore, endpoint broker, or remote Plan transport in the snippet.

THE IMPORTANT SEPARATION

Hold can recover managed local backup state. It cannot undo a remote machine replacement after the recipient observes it.

EVIDENCE / Outbox cancellation applies before claim. Remote undo is a non-goal.

RECEIPTS, NOT OMNISCIENCE

The agent gets a narrow command surface and correlated evidence.
Receipts record operational fact, not semantic correctness.

SOURCE / src/agent-cli.ts / src/runtime/Runtime.ts / test/parity-process.test.ts

THE AGENT SURFACE

RUNNABLE

```
1 airlock-agent doctor
2 airlock-agent actions
3 airlock-agent schema process.run
4
5 airlock-agent run workflow.air --compact
6 airlock-agent runs --limit 10
7 airlock-agent run-receipt --plan-id pln_...
```

ABRIDGED EXECUTED OUTPUT

PROOF EXCERPT

```
1 {
2   "schemaVersion": "airlock/program-run/v1",
3   "profile": "compatibility",
4   "workspace": "/tmp/airlock-proof/process/workspace",
5   "result": {
6     "state": "succeeded",
7     "result": {
8       "selected": "2:needle second\n",
9       "transformed": "2:NEEDLE SECOND\n",
10      "literal": "literal; touch should-not-exist",
11      "states": [
12        "succeeded",
13        "succeeded",
14        "succeeded"
15      ]
16    },
17    "counts": {
18      "plans": 3,
19      "actions": 3,
20      "artifacts": 6
21    }
22  }
23 }
```

PROJECTION BOUNDARY

This is a selected field projection from captured compact CLI output, not a standalone Schema-valid document. Program success still requires inspecting nested process state and outcome.

[OK] SUCCEEDED

All requested current actions completed and returned evidence.

[X] FAILED

Typed failure with completed records retained.

[~] PARTIAL

Earlier actions remain complete; no implied rollback.

REPLAY IS SEALED

A persistent run journal claims Plan identity before world work. Prior or concurrent execution rejects replay.

EVIDENCE BOUNDARY

Receipts show admitted, attempted, observed, staged, or changed work. They do not prove that opaque code fulfilled intent.

EVIDENCE / A durable receipt for every possible crash point remains an acceptance gate.

WHAT THE EVIDENCE EARNS

Airlock already covers meaningful agent scripting shapes. Strong shell-replacement confidence still depends on representative workload and adversarial evidence.

SOURCE / README.md / docs/acceptance.md / docs/evidence/parity-50.md

REPRODUCIBLE REPOSITORY GATES

EVIDENCE COMMANDS

```
1 bun run verify
2 bun scripts/prove-vouch.ts
3 bun scripts/prove-vouch-operations.ts
4 bun scripts/prove-parity-50.ts
```

VITEST FILES

52

+ 1 skipped

VITEST TESTS

255

+ 16 skipped

PARITY PROGRAMS

11

checked-in current set

REPEAT EXECUTIONS

50

10 cases x 5

WHAT WORKS COMPREHENSIVELY ENOUGH TO STUDY

Structured file work, captured processes, artifact pipelines, bounded control, native edits, archives, local Git, builds, recoverable recursive removal, and staged HTTP intent.

WHAT REMAINS BEFORE A STRONG CLAIM

Representative model-generated tasks, direct shell A/B evidence, wider crash/overlap injection, hostile closure tests, labels, endpoint brokerage, real remote Vouch, signing, and release provenance.

A STRONG FOUNDATION IS VISIBLE. THE RELEASE CLAIM REMAINS A GATE, NOT A SLOGAN.

READ NEXT

DESIGN.md / docs/macOS-v1.md / docs/security-model.md
docs/acceptance.md / examples/parity / examples/vouch

